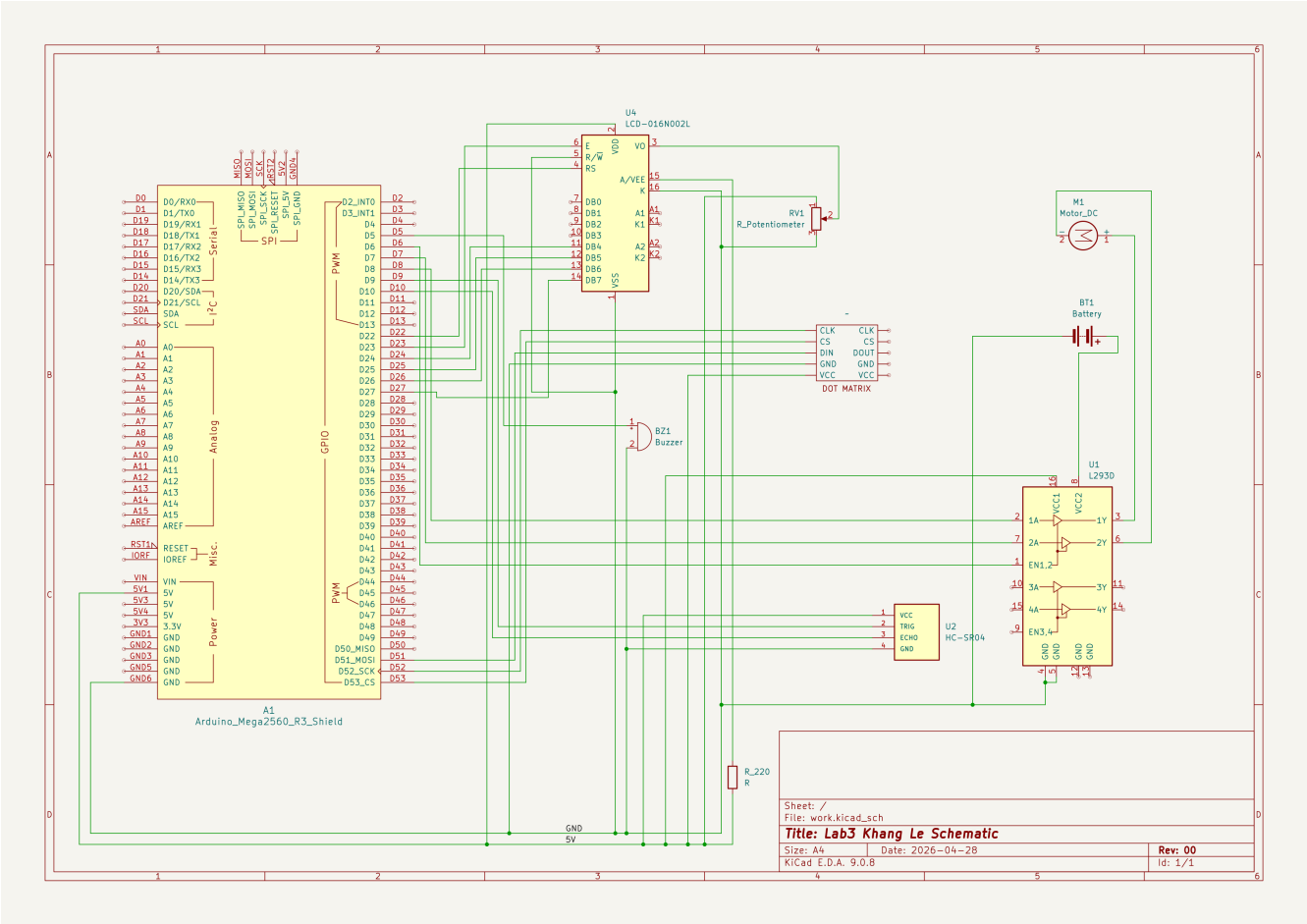
Student Name: Khang Le
Student ID Number: 01986773
Team Number: Myself
Team Members: Myself
Date of Completion: 2026-04-28
Demonstration Method: Through a video per professor permission.

Lab 3

1. Design

1.1 Hardware Design

Again, it is very nice that the components wiring is all given. Here is schematics. You can find the schematic source files along with higher quality render in `kicad` folder.



I don’t have L298N H-bridge module. I have neither the IC nor the daughterboard for the ease of wiring. The lab description says I should use them. But, I only have L293D. The wiring cannot be

exactly the same as the lab suggests so I have to open up the documentation of L293D to wire correctly. Instead of wiring 6 pins, I have the raw 16 pins instead.

Then, it also comes to me that I should not use Arduino board for powering the motor. I need external power for the motor to draw enough current. Otherwise, I would risk frying my board. So, I bought batteries and holder for the project. It works out good. I had fun playing with the motor for the whole day.

Also, I am glad that I have spare bread boards (by buying them). One single long bread board would not fit everything for this project. But, at the end, I don't have enough long wires to comfortably set up either ultrasonic sensor or the motor. It is just very awkward to have a good vantage point for these two components. Ultrasonic sensor requires clear view, which I don't have due to wires. So, what I did is to give it a pedestal, sort of, to rectify its visual occlusion and hereby improve vertical elevation vantage point for its sonic emanations. As for the motor, I just let it hang because I don't have wires. Luckily, none of this hinders the demonstration of the project.

And when I run the build, for some reasons, the electricals or something else behaves extremely weird. The buzzer does not output continuous beep but discrete and enunciated sound. The ultrasonic sensor drops distance to 0 randomly. I am not quite sure what the problem is. There are many things that could be the problem, like the power source or the faulty board or the motor.

1.2 Software Design

"Instructions on how to setup the project is very detailed and sufficient. All of the work is to fill in the blank. And funny enough, all the answers are already given. I only need to translate natural language into programming language."

I wrote that for lab2, I think it applies here as well.

Most of the code is not even programming the behavior of the build because the code is already made. The only thing I have to do for the Python TODOs is verifying that the output of the build matches what it is expected to do. And there are like 2 TODOs in Python code.

The only part that requires me to put some effort is step5 where I have to write my own protocol for data de/serialization. Personally, I think I have an advantage here because this is my 100th time writing de/serialization code. Overall, not very bad. And funny enough, I can check the `sensor_reader.py` file to see how the code should be implemented because that code does exactly what I am asked to do.

Serial Protocol Framing Report

* What start byte did you choose and why? I choose 0xAA because it is the default value. I don't think the header byte would matter because our data is hardcoded to be 7 bytes and the transmission is assumed to be reliable. To be honest, I don't think the start byte is even needed.

* What checksum algorithm? (XOR? Sum mod 256? CRC-8?) I use XOR checksum because it is the simplest.

* What happens if the start byte value appears in the payload? If the transmission is assumed to be reliable, nothing. The transmission is reliable, a frame is 7 bytes. Every read should guarantee a proper message. That means, no worry about interpreting wrong start byte.

* How would you make the protocol more robust? Turns it to TCP. With that, the datagram sending order is guaranteed and recovery from missing package is a problem. If that is unfeasible, I think the datagram could use some data recovery mechanism on top of checksum. With that, the reader can independently recover lost data.

1.3 Result

The lab3 repository is here: https://github.com/khanghugo/eece5520-lab1 (ignore the repo name)

The demonstration is here: https://youtu.be/884Vx6nzvPk

Everything works, except for the mysterious beeper and sensor problem. The beep is not continuous. The ultrasonic sensor drops distance to 0 sometimes. I look through the wiring, I don't see anything that would be a problem. I check the Arduino code, I don't think it is a problem either. But besides that, everything works when it works.

I will now answer questions listed in `docs/lab-writeup.md`

Step 1.

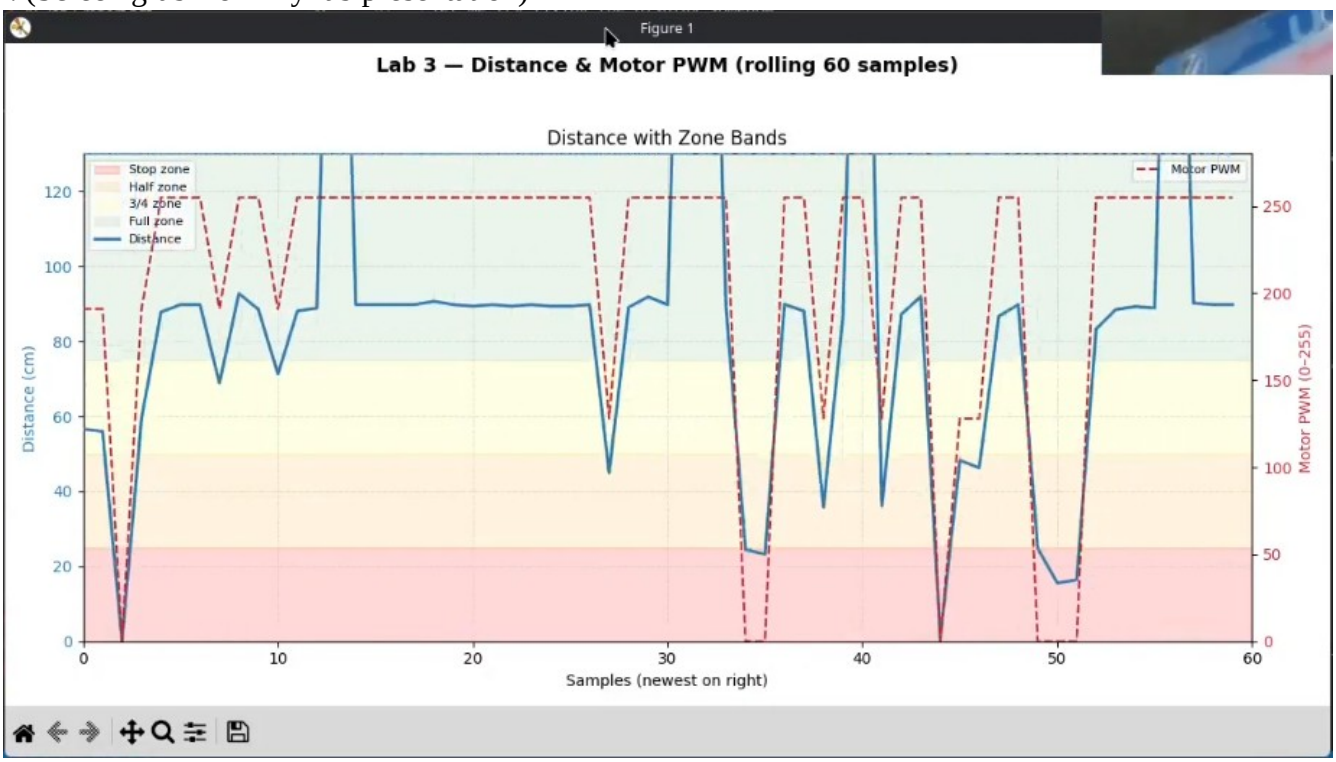
1. From Adafruit website, they say it works 2cm to 400cm away. The timeout is to make sure the sensor does not block the main procedure by keep waiting for an echo. 30_000us is the time limit for an echo to return. Then after that, the main procedure can run and the sensor can try listening again in the next loop. For that 30_000us and for the speed of sound (through air) be 343m/s, the distance sound moves in that 30_000us is 343m/s * 0.030s = 10.30m. But that is the whole round trip time, rather than the distance to the object, which is halved, at ~5m. That is the max distance for that timeout. It can be even farther from what I see in one of the Youtube video where the max distance goes up to 800cm.

2. Longer the signal, the more "signature" it is. It doesn't have to be 10us but it is what the example code from Elegoo is using. If it is shorter, it is possible that the trigger could not propagate properly, leading to possible missed signal. Longer signal just helps with ensuring the signal is there.

3. It is that the whole duration is from the trigger triggering a wave, the wave moves, the wave hits object, and the wave moves back. This duration includes both the duration of the wave moving and the wave moving back. By dividing by 2, we only get the distance where the wave hits an object, rather than the wave returns.

Step 2.

1. (Screen grab from my lab presentation)



They are proportionally linear. It should be because the bigger the distance, the more the fan spins, the higher duty cycle of PWM.

2. I expect the motor to run very slow or straight up not running due to being undervolt. At least this is not concerning of current, which can straight up cook the board. What I would do to fix it is to wire VCC2 to external/dedicated battery/power source while keeping the board logic at 5V.

3. Even though modern computer is very fast, this is just to make sure that plotting does not take too much CPU time and delays data acquisition. If data acquisition is delayed, it makes the graph not correct.

Step 3.

1.

At 50cm, the zone should have PWM=191. And yes, both Python and Arduino implementations agree.

```
[SensorReader] Port /dev/ttyACM0 opened — w
[SensorReader] Connected on /dev/ttyACM0 at
    56.4        191         191  ✓
    48.6        128         128  ✓
    49.7        128         128  ✓
    49.5        128         128  ✓
    55.1        191         191  ✓
    49.5        128         128  ✓
    51.6        191         191  ✓
    50.2        191         191  ✓
    50.8        191         191  ✓
    50.4        191         191  ✓
    50.9        191         191  ✓
```

Python code

```python
# ==========================================================================
# Reference implementation (used for comparison plot — do not modify)
# ==========================================================================

def _reference_distance_to_pwm(distance_cm: float) -> int:
    """Reference implementation for comparison."""
    if distance_cm > 75:
        return 255
    elif distance_cm > 50:
        return 191
    elif distance_cm > 25:
        return 128
    else:
        return 0
```

Arduino code

```arduino
// ========================================
// getPWM()  — provided helper
// ========================================
int getPWM(int zone) {
  switch (zone) {
    case 0: return 255;
    case 1: return 191;
    case 2: return 128;
    default: return 0;
  }
}
```

```arduino
// ========================================
// getZone()  — provided
// ========================================
int getZone(float dist) {
  if (dist > ZONE0_THRESHOLD) return 0;
  if (dist > ZONE1_THRESHOLD) return 1;
  if (dist > ZONE2_THRESHOLD) return 2;
  return 3;
}
```

```
// ── Zone thresholds (cm) ──────────
#define ZONE0_THRESHOLD  75
#define ZONE1_THRESHOLD  50
#define ZONE2_THRESHOLD  25
#define BUZZER_SILENT_THRESHOLD  60
#define BUZZER_ALARM_THRESHOLD   20
```

The code looks very similar to me. Both code just do the same thing but Arduino code just decides to use static variables and helper methods.

2.

I will be talking about Python code because it is basically one function. I will just add another `elif` condition and then give the PWM value. For that example, 65-75cm at 90% speed, I will add

```
elif distance_cm > 65:
        return int(255 * 0.9)
```

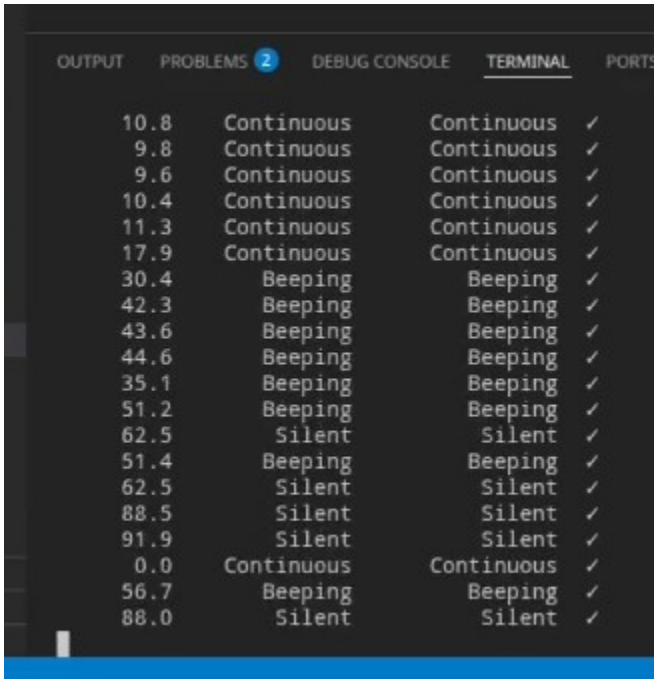
right after the first `if` branch

3.

I don’t think this premise is true. I think continuous proportional is a lot easier because the implementation is based on mathematical presentation. Less code to write in comparison with defining each zone. But I guess zone-based is more predictable and the behavior is simpler. The motor will change its speed less often. But the downside is choosing what pre-defined value is to have a good default.

Step 4.

1.



OUTPUT PROBLEMS 2 DEBUG CONSOLE TERMINAL PORTS

10.8	Continuous	Continuous	✓
9.8	Continuous	Continuous	✓
9.6	Continuous	Continuous	✓
10.4	Continuous	Continuous	✓
11.3	Continuous	Continuous	✓
17.9	Continuous	Continuous	✓
30.4	Beeping	Beeping	✓
42.3	Beeping	Beeping	✓
43.6	Beeping	Beeping	✓
44.6	Beeping	Beeping	✓
35.1	Beeping	Beeping	✓
51.2	Beeping	Beeping	✓
62.5	Silent	Silent	✓
51.4	Beeping	Beeping	✓
62.5	Silent	Silent	✓
88.5	Silent	Silent	✓
91.9	Silent	Silent	✓
0.0	Continuous	Continuous	✓
56.7	Beeping	Beeping	✓
88.0	Silent	Silent	✓

I cannot do 19.9 but from the number 17.9 up there, it should be continuous.

2.
The beep frequency increases because of the “map” function. What it does is that it map a range to another range. It maps distance range 20cm-60cm to audio frequency 2000hz-500hz. This means, at distance 20cm, I get sound of 2000hz. At distance 60cm, I get sound of 500hz. For my personal user experience, I become more attentive as the distance closes because the sound is very annoying.

3.
I think it is that I want to know IF I am close or not close, rather than HOW close I am. Basically, I don’t want much feedback. I only care about the outcome (in a sense). Variable frequency is important when every single feedback is needed because the user wants to know HOW close they are.

Step 5.

1.
If there is a missing byte, the start byte is out of order in the stream. It is possible to discard some bytes until start bytes. But then, it is also possible for packets inside the frame to have the same value as the missing byte. I don’t think making the parser robust is the solution. I think making the data transfer robust is the solution. I think the parser should work with good data to produce good result. And it is up to the transmission mechanism to guarantee that.

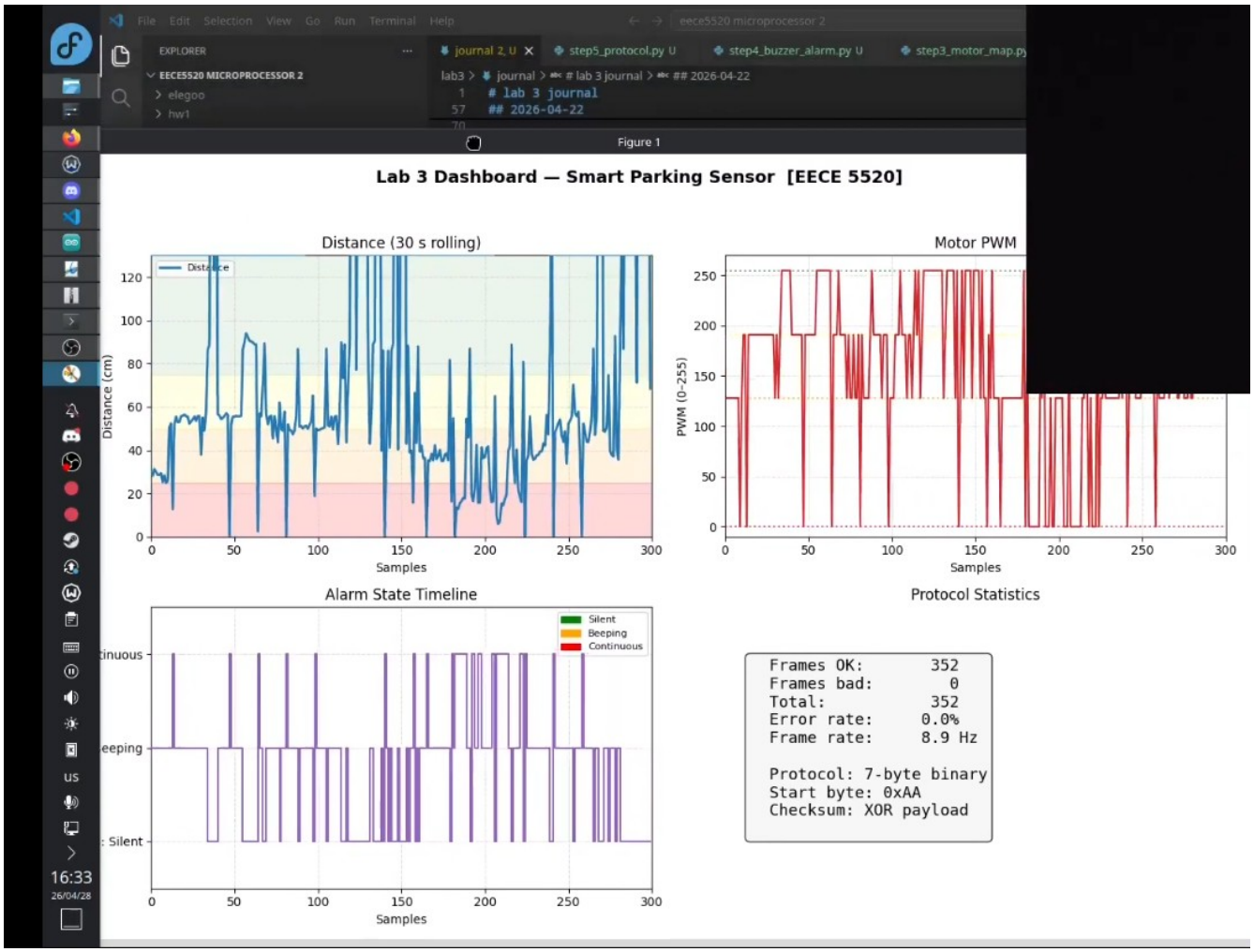
2.

First of all, the parser would not know that byte belongs to the distance field. So, more likely, it would interpret that distance field high byte as a new frame, and that will lead to corrupted reading. To fix it, it is better to buffer the data, and then look ahead from each 0xAA byte to determine a frame.

3.
From a Google search, Sum Mod 256 can “carry propagation” so it is better to detect errors. Both are very simple to implement. But for the actual circuitry/assembly complexity, Sum Mod 256 takes a lot more cycle to execute. When implementing them on higher level programming language like Arduino C++ or Python, they are equally easy to write.

Step 6.

1.



It is 8.9Hz for me. So it does not match the expected 10Hz. Since the ECHO hold time is 30ms, this could be the reason for it. It adds up. Also, sending data through serial could also contribute to it.

2.
I think the checksum error could be from data loss, making one of the packet dropped, resulting in half a frame or so and the code has to work with bad data. I did not have any error after running the code for 30 seconds.

3.
Make it TCP. I think the solution is to make data transmission reliable. And as far as I know, the issue is solved and the solution is TCP. Or, there could be some kind of data recovery scheme to recover dropped packet done by the receiver.

2. Problems Encountered and Solved

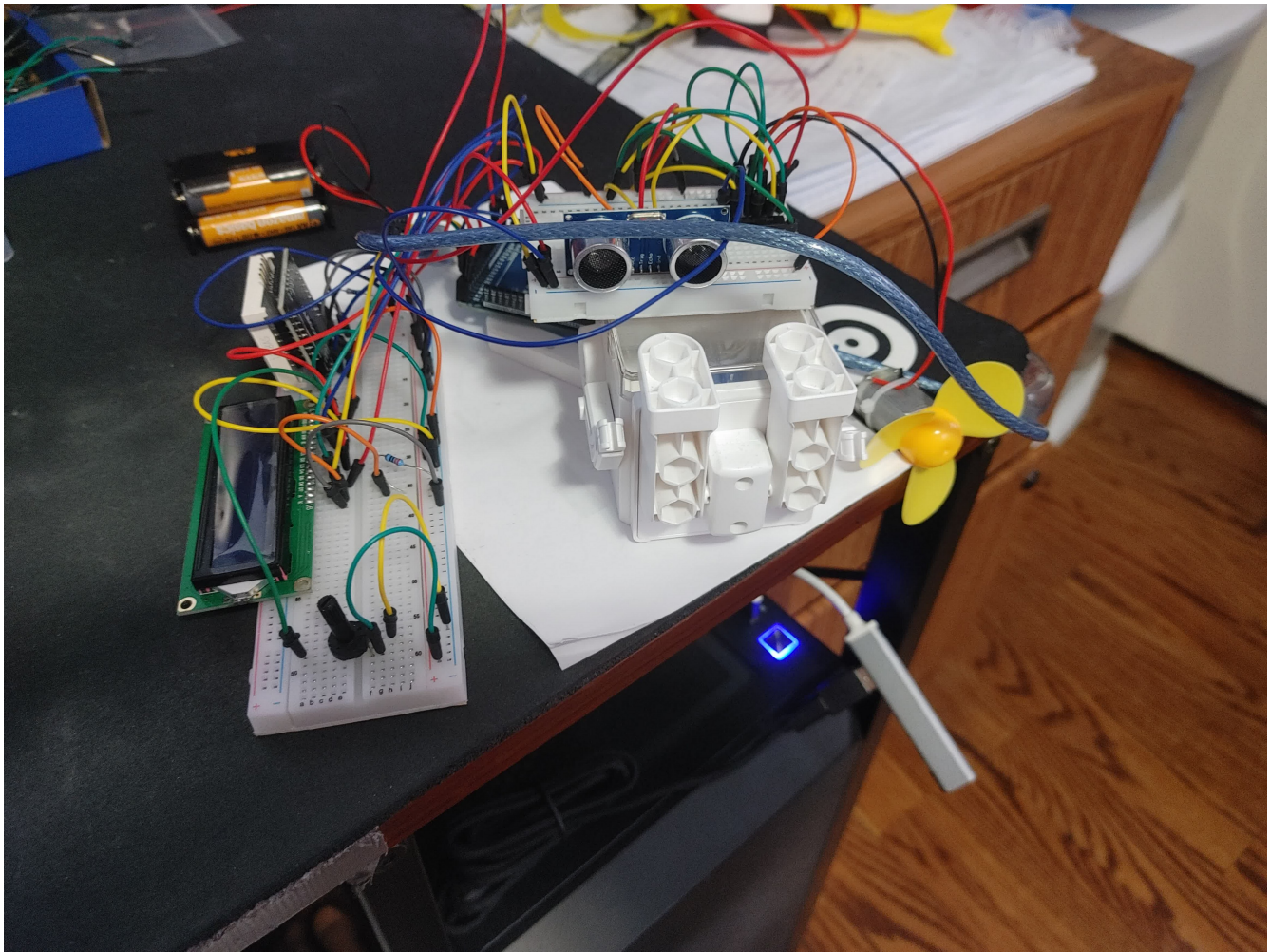
Some small problems with the project. First, I don’t have L298H bridge as suggested by the labs documentation. I only have L293D and I don’t even have the daughter board to help simplify wiring. But, I am able to read the documentation and wire it myself based on the suggested wiring included in the Arduino file.

I also have a small problem with writing the protocol. It is fairly minor. It is that I forgot to convert the measured distance during deserialization. If I just ignore the measure distance multiplication/division, the de/serialization code are perfectly inverse. I took me longer than it should to realize that I had to divide the number again.

Also, slightly embarrassing, I was mistaking a rotary encoder for a potentiometer.

3. Personal Contribution to the Lab

I did all the work by myself. More images in the `lab3/shots` folder



4. Lessons Learned -- Conclusion

I learn how to use ultrasonic sensor today and how it should work. I watched a Youtube video on the sensor and it is nice to learn that it can fail! I rarely work with hardware and I only work with software. From what I know about software, it is very easy to make robust, reliable, and reproducible product. Once a software product is deployed, it is expected to work until the end of time. If there is a bug, it can be easily fixed. But for hardware, there is no such perfection. Everything is constrained and has tolerance of error.

Timer1 thing has already been discussed in lab1 for me. The problem with polling is that the procedure is running on the main procedure. It takes CPU cycle to poll the device. Interrupt is a nice mechanism to avoid that since it is built into the hardware and basically allows asynchronous execution. But this usually applies to limited environment like the low compute power of a microcontroller. For bigger chips, they can handle extra cycles just fine.

I think this build could be improved further yes. The sensor could be wireless, totally detached from a board so that I can put it anywhere I like. I think using a camera is nice too. It could somehow enhance the ultrasonic senor. But maybe I will leave that to my future self who might be interested enough to carry it out.